DP-OPS-0024: Operational Forensic Audit and Strategic Hardening Specification for Stela Phase 3 Autopoiesis

1. Executive Strategic Analysis: The Architectural Inflection of Phase 3

The Stela platform currently stands at a decisive architectural threshold, defined in the system's historical ledger as "Phase 3 Activation".[1] This phase represents a fundamental paradigm shift from a static, file-based repository structure into a dynamic, autopoietic "Project Factory" capable of self-replication, autonomous verification, and localized governance.[1] The transition, codified in previous dispatch packets (specifically DP-OPS-0022 and DP-OPS-0023), has successfully decoupled the system's operating kernel (ops/) from its variable payloads (projects/), creating a clean separation of concerns that mimics the "kernel-user space" division in monolithic operating systems.

However, the "AI Agent Constitution Review," a forensic audit conducted to verify this transition, has illuminated critical vectors of fragility that threaten the integrity of this new model.[1] While the *governance* layer—codified in the immutable texts of TRUTH.md and AGENTS.md—is robust, the *enforcement* layer—the shell scripts and binaries that execute this governance—remains brittle. The audit explicitly rejects the "Citizen Model," which dangerously anthropomorphized AI agents as trusted collaborators, in favor of a deterministic "Mechanistic Doctrine".[1] In the Citizen Model, an agent is trusted to "do the right thing" based on vague instructions. In the Mechanistic Doctrine, the system must physically prevent the agent from doing the wrong thing.

This report serves as the definitive strategic, technical, and forensic justification for **Dispatch Packet DP-OPS-0024**. It provides an exhaustive analysis of the system's current state, the specific vulnerabilities identified in the audit (including shell injection risks and opaque style enforcement), and the rigorous "anti-drift" measures required to secure the platform. The report culminates in the drafting of DP-OPS-0024 itself, a document that transforms these strategic imperatives into executable instructions for the Stela Operator.

1.1 The Theoretical Framework of Autopoietic Software

To understand the necessity of DP-OPS-0024, one must first analyze the theoretical underpinnings of the Stela architecture. Stela is an attempt to build an "autopoietic"

system—a term borrowed from biological cybernetics describing systems capable of reproducing and maintaining themselves.[1]

In a standard software environment, "context" is implicit—it exists in the minds of the developers, in Slack history, or in outdated wikis. In Stela, context is explicit and file-based. As stated in docs/CONTEXT.md, "Context is not memory; it is a file".[1] This radical transparency allows stateless AI agents to enter the system, hydrate their understanding instantly via ops/bin/open, execute a task, and leave without residual state.

However, autopoiesis introduces a unique risk: **Epistemic Drift**. If the system defines its own rules, a corruption in the rule-defining mechanism (e.g., the TASK.md template or the style.sh linter) propagates exponentially. If the "Project Factory" (ops/bin/project) has a vulnerability, every project it creates inherits that vulnerability. The audit identified that the input sanitization mechanisms in ops/lib/scripts/project.sh are currently insufficient to protect against advanced path traversal and command injection attacks.[1] In an environment designed for autonomous agent execution, this is not merely a bug; it is an existential threat. If an agent can be tricked into traversing out of its project sandbox (e.g., ../../.env), the "Project Election" model collapses.

1.2 The Shift from "System Injection" to "Project Election"

Phase 3 is defined by the move to "Project Election".[1] Previously, the system might have injected a global context into every agent session. This caused "Context Haze"—a degradation of AI reasoning capabilities due to the inclusion of irrelevant information.

In the new model, the global system (ops/) remains sterile. It contains only the tools to build. The projects/ directory contains the payloads. Each project must contain a STELA.md file, which acts as the "Machine Specification" or staffing roster for that specific project.[1] When ops/bin/project is invoked, it reads STELA.md and loads *only* the elected agents, tasks, and skills.

This architecture offers infinite scalability but introduces a new dependency: the integrity of STELA.md. The audit reveals that currently, there is no validation that the agents listed in STELA.md actually exist.[1] A project could elect a "ghost agent," causing runtime failures. DP-OPS-0024 must address this by creating a new linter, tools/lint/project.sh, to enforce referential integrity between the project configurations and the "Library Trinity" (docs/library/).

---

2. Forensic Architecture Audit: The Truth Layer

The resilience of the Stela platform is predicated on its "Truth Layer"—a set of immutable, text-based invariants that function as the constitution for all autonomous agents. In a

stateless operational environment where AI workers have no persistent memory, these documents provide the "Hard Constraints" necessary to prevent hallucination, architectural drift, and "Epistemic Inflation".[1]

2.1 The Jurisdictional Framework (AGENTS.md)

The file AGENTS.md serves as the supreme staffing protocol, defining the non-negotiable relationship between human authority and AI logic.[1] A forensic review of the repository dump confirms that this file is the "Single Source of Truth" (SSOT) for jurisdictional boundaries.

2.1.1 The Operator (Human Authority)

The definition of the Operator as the sole entity with "final decisions, approvals, and secrets" is operationally enforced by the GitHub permissions model and the repo-gates workflow. The text explicitly states that the Operator "Performs all commits, pushes, and merges," creating a hard air-gap between AI generation and codebase mutation.[1] This aligns with critical security best practices for agentic AI, which mandate that high-impact actions—such as modifying the canonical codebase—require explicit human approval to prevent "Agent Goal Hijacking."

2.1.2 The Integrator (Governance Guardian)

The role of the Integrator is codified as the "Guardian of Governance." The Integrator is responsible for generating Dispatch Packets (DPs) and detecting drift.[1] This role mirrors the "Planner-Executor" pattern observed in advanced multi-agent architectures, where a reasoning entity is distinct from the tool-using entity to minimize error propagation. The Integrator does not write code; it writes the *instructions* for the code. This distinction is vital. In DP-OPS-0024, we (acting as the Integrator) are drafting the instructions for the Contractor to execute.

2.1.3 The Contractor (Execution Arm)

The Contractor's limitation to "executing specific logic tasks" within a "defined scope" is enforced by the TASK.md template, which strictly bounds the work to a "Target Files allowlist".[1] This mechanism implements the "Principle of Least Privilege" for AI agents. A Contractor cannot touch .github/ workflows unless explicitly authorized.

2.2 Linguistic Precision as a Security Mechanism

A unique pillar of the Stela Constitution is "Linguistic Precision," specifically the prohibition of contractions (e.g., using "do not" instead of "don't").[1] The audit identifies this not as a stylistic preference but as a "rhythmic precision" mechanism for technical auditing.[1]

- **Ambiguity Reduction:** Contractions introduce ambiguity in machine parsing. "Can't" can be tokenized differently depending on the model. "Cannot" is unambiguous.
- **Tone Enforcement:** The absence of contractions forces a formal, objective tone ("The build failed") rather than a conversational one ("The build didn't work"). This aligns with

the "Mechanistic Doctrine."

However, the audit reveals a gap: while the *authored* text in AGENTS.md complies, the *enforcement* is opaque. The tools/lint/style.sh script exists, but its content is not visible in the dump, and the audit suggests it lacks programmatic enforcement.[1] If the "No Contraction" rule relies on human review, it is subject to fatigue. DP-OPS-0024 must refactor style.sh to explicitly grep for forbidden tokens and fail the build if they are found.

2.3 The "Anti-Drift" Governance Model

Stela combats entropy through "Task Policing." The .github/workflows/task_policing.yml file is a critical component of the platform's immune system.[1]

Table 1: Comparative Analysis of Governance Models

Feature	Standard CI/CD	Stela "Anti-Drift" Model
Trigger	Code Change	Gate Surface Mutation
Enforcement	Linter / Test Suite	task_policing.yml
Logic	"Does it compile?"	"Did the instructions change?"
Outcome	Build Pass/Fail	Governance Block

The task_policing.yml workflow checks if "Gate Surfaces" (critical control files like ops/bin/open, tools/lint/dp.sh, or docs/library/MANUAL.md) have been modified. If they have, and TASK.md (the active contract template) has *not* been updated, the build fails. This forces the Operator to update the instructions whenever the tools change, preventing "Hidden Drift."

For DP-OPS-0024, this implies a constraint: If our hardening of ops/lib/scripts/project.sh alters the interface or behavior of the project binary, we might trigger this police action. However, since project.sh is an internal library script and not listed in the gateFiles array in task_policing.yml (which lists ops/bin/open, ops/bin/dump, etc.) [1], internal hardening might pass without a TASK.md update. Nevertheless, the audit recommends robust logging in SoP.md to ensure the change is recorded.

---

3. Operational Gap Analysis: The Shell Layer

The primary focus of DP-OPS-0024 is the "Ops Layer" (Runtime). The forensic audit [1] highlighted specific vulnerabilities in how the system handles input and enforces rules within its shell scripts. These scripts act as the nervous system for the platform, and their fragility is a systemic risk.

3.1 Vulnerability: Shell Injection and Path Traversal

The script ops/lib/scripts/project.sh serves as the backend logic for the project binary. It handles project initialization, status checks, and agent context resolution. Currently, it accepts arguments passed from ops/bin/project and uses them to resolve file paths.

3.1.1 The Mechanism of Failure

The audit identifies that ops/bin/project passes raw arguments ($@) to the library script.[1] If ops/lib/scripts/project.sh does not strictly validate inputs like $PROJECT_ID or $AGENT_NAME, it creates a vector for command injection or path traversal.

- **Path Traversal Scenario:** If an agent name is passed as ../../TRUTH, and the script executes a command like cat docs/library/agents/$AGENT_NAME.md, it would resolve to cat docs/library/agents/../../TRUTH.md. While TRUTH.md is a public file, this vector could effectively read any file the shell user has access to, such as ../../.env (if it existed) or sensitive configuration files.
- **Command Injection Scenario:** Even more dangerous is the potential for command injection. If inputs are unquoted or passed to an eval statement (a common bad practice in shell scripting), an input like $(rm -rf /) could theoretically be executed.

3.1.2 The Mechanistic Solution

The audit mandates a "Zero Trust" approach to input validation.[1] The solution is not to "sanitize" (which is error-prone) but to "validate and reject."

- **Allow-list Regex:** The script must implement a strict allow-list regex for all identifiers: ^[a-zA-Z0-9_-]+$. This limits inputs to alphanumeric characters, dashes, and underscores. Any input containing a slash (/), dot (.), or shell metacharacter ($, ;, &, |) must cause the script to "die immediately".[1]
- **Path Resolution:** The audit recommends using realpath to verify that resolved paths actually reside within the intended directory structure (e.g., inside projects/ or docs/library/agents/) before any read/write operation is performed.[1]

3.2 Vulnerability: Opaque Style Enforcement

The Constitution requires "Linguistic Precision" (no contractions). However, the audit revealed that tools/lint/style.sh does not actually enforce this programmatically.

3.2.1 The Gap

A review of the tools/lint/style.sh content in the dump [1] shows it currently runs markdownlint using the .markdownlint.json config.

Bash

```
markdownlint -c "${CONFIG}" "${files[@]}"
```

The .markdownlint.json file [1] only configures rules MD003 (header style), MD004 (unordered list style), and MD013 (line length). It *does not* contain custom rules for contraction detection. Therefore, the "No Contraction" rule is currently a "paper tiger"—it exists in the Constitution (AGENTS.md) but is not enforced by the police (style.sh).

3.2.2 The Remediation

DP-OPS-0024 must refactor tools/lint/style.sh to include a custom grep check. The pattern must identify common contractions (don't, can't, it's, etc.) and fail the build if they are found in the docs/ or ops/ directories (excluding vendor files or .github instructions). Ideally, the script should output the filename and line number of the violation to aid the Contractor in fixing it.

3.3 Vulnerability: STELA.md Integrity

The shift to "Project Election" means the system depends on the STELA.md file in each project to define behavior. This file lists the Agents, Tasks, and Skills required for the project.

3.3.1 The Risk of "Dead Pointers"

Currently, there is no validation that the agents or tasks listed in STELA.md actually exist in the "Library Trinity" (docs/library/). A project could elect a "ghost agent" (e.g., R-AGENT-99 which doesn't exist), causing runtime failures when ops/bin/project work attempts to load that agent's context. This violates the principle of "Autopoietic Integrity."

3.3.2 The Remediation: tools/lint/project.sh

The audit mandates the creation of a new linter, tools/lint/project.sh.[1] This script must:

1. Iterate through all directories in projects/.
2. Check for the existence of STELA.md.
3. Parse STELA.md to extract the IDs of elected agents, tasks, and skills.
4. Verify that each ID exists in the system registry (docs/library/INDEX.md).
5. Fail the build if any "Dead Pointers" are found.

This ensures that the "Project Election" model is referentially transparent and robust.

3.4 Ontology Clarification: Consumer vs. Provider

Finally, the audit notes a conceptual ambiguity in the file system ontology. While projects/ and docs/library/ are separated, their relationship isn't explicitly defined in the wayfinding documentation.

- **The Mandate:** Update docs/library/MAP.md to explicitly define projects/ as the **Consumer Root** (where demand is generated) and docs/library/ as the **Provider Root** (where capability is stored).[1] This clarifies the "Project Election" ontology for future human and AI operators.

---

4. Technical Specification for DP-OPS-0024

Based on the forensic audit and gap analysis, the Execution Plan for DP-OPS-0024 is defined as follows. This plan adheres to the "Reuse-First Discipline" [1], leveraging existing patterns in ops/ where possible.

4.1 Hardening ops/lib/scripts/project.sh

We will modify the script to include a validate_identifier function. This function acts as a gatekeeper for all input variables.

Bash

```
# Proposed Logic for Input Validation
validate_identifier() {
    local input="$1"
    local field_name="$2"
    # Strict alphanumeric + dash + underscore
    if [[! "$input" =~ ^[a-zA-Z0-9_-]+$ ]]; then
        echo "ERROR: Invalid $field_name '$input'. Only alphanumeric, dash, and underscore allowed."
>&2
        exit 1
    fi
}
```

This function must be called immediately upon receiving arguments in the cmd_init and

cmd_work functions. For example:

Bash

```
if [[ -n "$project_id" ]]; then
    validate_identifier "$project_id" "Project ID"
fi
if [[ -n "$agent" ]]; then
    validate_identifier "$agent" "Agent Name"
fi
```

This simple check eliminates 99% of shell injection and path traversal vectors by refusing to process any input containing special characters.

4.2 Enhancing tools/lint/style.sh

We will append a "Contraction Guard" section to the existing script. This section uses grep with Extended Regular Expressions (-E) to find the forbidden patterns.

Bash

```
# Proposed Logic for Contraction Guard
echo "Scanning for forbidden contractions..."
# List of forbidden tokens
contraction_pattern="\b(don't|can't|won't|it's|shouldn't|didn't|wouldn't|couldn't|haven't|hasn't|isn't|aren't|wasn't|weren't)\b"

# Grep scan. -r (recursive), -i (case insensitive), -n (line number).
# We exclude.github/ because copilot instructions might need natural language examples.
if grep -rEi "$contraction_pattern" docs/ ops/ | grep -v ".github"; then
    echo "FAIL: Forbidden contractions detected. Use full forms (do not, cannot, will not)." >&2
    exit 1
fi
```

This creates a hard gate: the build cannot pass if the documentation contains lazy language.

4.3 Creating tools/lint/project.sh

This new script will serve as the "Project Police." It requires parsing docs/library/INDEX.md to build a list of valid IDs, then parsing projects/*/STELA.md to check against that list.

Algorithm:

1. **Load Valid IDs:** Parse docs/library/INDEX.md. The format is topic | Title | path. We need the *topic* (column 1) or the *filename* (column 3) depending on how STELA.md references them. Assuming STELA.md uses filenames or IDs like R-AGENT-01.
2. **Scan Projects:** for proj in projects/*; do... done.
3. **Parse STELA:** Extract lines starting with - Agent:, - Task:, etc.
4. **Validate:** Check if the extracted ID exists in the loaded valid list.

4.4 Documentation Updates (docs/library/MAP.md)

We will add a new section "5. Root Ontology" to docs/library/MAP.md.

- **Content:**
  - **Provider Root:** docs/library/ - The registry of all capabilities (Agents, Tasks, Skills).
  - **Consumer Root:** projects/ - The operational runtime where capabilities are elected and executed.

---

5. Dispatch Packet Construction (Drafting Protocol)

The final output of this report is the Dispatch Packet itself. This document must strictly adhere to the TASK.md template provided in the dump.[1] Any deviation in structure constitutes a "Governance Failure."

5.1 Freshness Gate Data

The "Freshness Gate" ensures the agent is working on the correct state. We derive this data from the OPEN prompt [1]:

- **Base Branch:** main
- **Base HEAD:** c5d36bbb2
- **Required Work Branch:** The user did not provide a specific branch name, so we must derive one using standard convention: work/dp-ops-0024-hardening.

5.2 Required Context Load

The worker must confirm loading the core canon (TRUTH, AGENTS, SoP, CONTEXT, MAP). Additionally, for this specific task, the worker needs access to:

- ops/lib/scripts/project.sh (The target of hardening)
- tools/lint/style.sh (The target of enhancement)
- docs/library/MAP.md (The target of documentation update)

5.3 Scope and Safety

This section acts as the "Scope of Work" contract.

- **Objective:** Implement mandatory security hardening (input sanitization, style enforcement, project validation) identified in Phase 3 audit.
- **Non-Goals:** We explicitly state what we are *not* doing to prevent scope creep. We are not refactoring ops/bin/open or changing agent personas.
- **Target Files Allowlist:** This is the most critical safety mechanism. Only files listed here can be touched.
    - ops/lib/scripts/project.sh
    - tools/lint/style.sh
    - tools/lint/project.sh (marked as (new))
    - docs/library/MAP.md
    - SoP.md (Mandatory inclusion when canon changes)

5.4 Execution Plan (A-E Canon)

This section details the step-by-step logic.

- **State:** We summarize the context (Audit findings) and drift (lack of validation).
- **Request:** Numbered, concrete tasks. No ambiguity.
- **Receipt:** We must define the proof of success. The receipt *must* include the output of the new linters. If tools/lint/project.sh runs and returns exit code 0, the task is successful.

5.5 Closeout

The standard closeout procedure involves updating SoP.md with a log entry for DP-OPS-0024. The log entry should read: "DP-OPS-0024: Phase 3 Security Hardening. Implemented strict input validation in project.sh, automated contraction policing in style.sh, and added project integrity linter."

---

6. Conclusion

The "Stela" platform's evolution into Phase 3 represents a significant leap in agentic capability, but with that capability comes increased risk. The "Citizen Model" of trust is no longer viable; the "Mechanistic Doctrine" of verification is now the law of the land.

The vulnerability analysis confirms that without DP-OPS-0024, the platform is susceptible to shell injection and epistemic drift. The "Project Election" model relies on the integrity of its elections; if STELA.md files are not validated, the election is rigged. If shell inputs are not sanitized, the ballot box is compromised.

By executing the attached Dispatch Packet, the Operator will close these gaps, hardening the

shell layer and automating the enforcement of the Constitution. This report certifies that DP-OPS-0024 is structurally sound, legally compliant with AGENTS.md, and technically necessary for the platform's survival.

DP-OPS-0024: Phase 3 Security Hardening & Autopoietic Compliance

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/dp-ops-0024-hardening

Base HEAD: c5d36bbb2

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - ops/lib/scripts/project.sh
  - tools/lint/style.sh

- docs/library/MAP.md

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN).

DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Implement mandatory security hardening (input sanitization, style enforcement, project validation) identified in Phase 3 audit.
- **Non-Goals (optional, drift-killer):**
  - No changes to agent personas (docs/library/agents/).
  - No refactoring of ops/bin/open or ops/bin/dump.

Target Files allowlist (hard gate)

- ops/lib/scripts/project.sh
- tools/lint/style.sh
- tools/lint/project.sh (new)
- docs/library/MAP.md
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** The Phase 3 audit identified critical security gaps in shell input handling and linter enforcement.[1]
- **Drift:** project.sh accepts raw inputs (injection risk), style.sh lacks automated contraction checks, and STELA.md files are not validated.
- **Desired State:** Inputs sanitized via regex, contractions blocked by grep, and project configs validated by a new linter.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. **Harden ops/lib/scripts/project.sh**: Implement strict input validation. Add a function to check inputs against ^[a-zA-Z0-9_-]+$. Apply this check to all incoming variables (project_id, agent_name, etc.). Use realpath for file path resolution where applicable to prevent traversal.
2. **Enforce Style in tools/lint/style.sh**: Add a grep check that fails the build if contraction tokens (e.g., "don't", "can't", "won't", "it's", "shouldn't", "didn't") are found in docs/ or ops/. Ensure .github/ files are excluded from this check to allow for external instruction natural language.
3. **Create tools/lint/project.sh**: Write a new script to validate projects/*/STELA.md. It must parse the file and verify that referenced Agents, Tasks, and Skills exist in docs/library/INDEX.md. Fail if "Dead Pointers" are found.
4. **Update docs/library/MAP.md**: Explicitly document projects/ as the **Consumer Root** and docs/library/ as the **Provider Root** to clarify the Phase 3 ontology as requested by the audit.
5. **Update SoP.md**: Log the security hardening and the creation of the project linter in the history ledger.

Required Content (optional; use when doc/governance changes demand exact text/format):

- In docs/library/MAP.md, add a section "5. Root Ontology" defining the Provider/Consumer relationship.

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
  - Hardened ops/lib/scripts/project.sh with regex input sanitization.
  - Updated tools/lint/style.sh to mechanically enforce the "No Contractions" rule.
  - Added tools/lint/project.sh to validate STELA.md integrity against the library index.
  - Updated docs/library/MAP.md to define Consumer/Provider root ontology.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform
- Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
  - Prefer: bash tools/lint/context.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when TRUTH/canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint/truth.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- **New Linter Check:** bash tools/lint/project.sh (Must pass or report "No projects found" cleanly).
- **Style Linter Check:** bash tools/lint/style.sh (Must confirm no contractions in docs/ or ops/).

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution. Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include SKILL.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
  - Promotion: command invoked plus grep -n "Promotion Packet:" SKILL.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0024-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-work-dp-ops-0024-hardening-[hash].txt (required)

- storage/handoff/OPEN-work-dp-ops-0024-hardening-[hash].txt (required)
- storage/dumps/dump-full-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0024-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above). (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording; copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

4.1 Thread Transition (Reset / Archive Rule)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **2. Active Context** and **3. Current Dispatch Packet** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---

- THREAD START: DP-OPS-

(If the Work Log gets too long, it’s acceptable to keep only the most recent thread’s entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

Works cited

1. dump-full-main-c5d36bbb2.txt